



COMSATS University Islamabad, Attock campus
Department of Computer Science

Program: BS-SE

Sp24-BSE-058

Sumbal

Assignment # 04

Information Security

06-December-2025

Ma'am Ambreen Gul

Task 1: Simple RSA Implementation (Without Libraries)

```
p = 11
q = 13
n = p * q
phi = (p - 1) * (q - 1)
e = 7

def mod_inverse(a, m):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

d = mod_inverse(e, phi)

print("Public Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))

# Encryption and Decryption
def encrypt(m, e, n):
    return pow(m, e, n)

def decrypt(c, d, n):
    return pow(c, d, n)

# ----- NEW PART: FULL SENTENCE -----
message = "HELLO"
print("\nOriginal Message:", message)

# Convert each character to ASCII → encrypt
encrypted_list = [encrypt(ord(ch), e, n) for ch in message]
print("Encrypted:", encrypted_list)

# Decrypt each number → convert back to characters
decrypted_list = [chr(decrypt(c, d, n)) for c in encrypted_list]
decrypted_message = "".join(decrypted_list)

print("Decrypted:", decrypted_message)
```

Code Explanation:

1. **Choose two prime numbers (p and q):**

RSA starts by selecting two prime numbers. These primes are the base of the security.

2. **Compute $n = p \times q$:**

This value becomes part of both the public and private key.

It is also used in encryption and decryption.

3. **Compute $\phi(n) = (p-1)(q-1)$:**

$\phi(n)$ represents how many numbers are relatively prime to n .

It is used to compute the private key.

4. **Select public exponent e:**

We choose a number e that has no common factors with $\phi(n)$.

It is used for encryption.

5. **Compute private exponent d:**

d is the modular inverse of $e \bmod \phi(n)$.

It is used for decryption.

This ensures:

$$(\text{message}^e)^d \bmod n = \text{message}$$

6. **Encryption:**

Message is converted into a number and then encrypted as:

$$C = M^e \bmod n$$

7. **Decryption:**

The original message is recovered using:

$$M = C^d \bmod n$$

Output:

```
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)

Original Message: HELLO
Encrypted: [19, 108, 54, 54, 40]
Decrypted: HELLO
PS C:\Users\hp\Desktop\python workspace>
```

Task 2: RSA with PyCryptodome:

```
# -----
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import binascii

# Generate 2048-bit RSA key pair
key = RSA.generate(2048)
public_key = key.publickey()

# Encrypt message
message = "Hello RSA from Student!"
cipher = PKCS1_OAEP.new(public_key)
encrypted = cipher.encrypt(message.encode())

# Decrypt message
cipher2 = PKCS1_OAEP.new(key)
decrypted = cipher2.decrypt(encrypted)

print("\n--- RSA 2048-bit Example ---")
print("Original Message:", message)
print("Encrypted (hex):", binascii.hexlify(encrypted).decode())
print("Decrypted:", decrypted.decode())
```

Explanation:

1. Generate a 2048-bit key pair:

PyCryptodome automatically creates a secure RSA key with a large size (2048 bits).

- The **private key** stays secret.
- The **public key** is shared publicly.

2. Create an RSA cipher object (OAEP):

OAEP is a secure padding scheme used for RSA encryption.

It prevents attacks on plain RSA.

3. Encrypt the message:

The public key encrypts the message.

The encrypted output is shown in **hex format** because it is binary data.

4. Decrypt the message:

The private key decrypts the encrypted text back to the original message.

5. Important note:

RSA with real cryptographic libraries uses large key sizes and secure padding to ensure real-world security

Output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
---- RSA 2048-bit Example ----
Original Message: Hello RSA from Student!
Encrypted (hex): 765596bd9d2fad84fd7efc29b00c592f96e48a7bc07388c721208be2c06d172db455558f3ce123e9a5a37c707df
80325f52208aea1f5f8345b7c76d1b08b6b12821f976a2f9cc38d30f5e199c8140701f92692fb3bbe013114e2215db54db3c5038d4fd
f6a82b33f0752c09b4beece18591920ef31aec80fb9dba440175eb1b916a8a2c290e1525073e2eb2e097b86ac6125693ab64d29aeb90
ff27d6361ae37db649a0451d8698a547b7a61deb74cbcf65b35344286a0e4604123e70b1e933a8e21e191205356b2109e6e004e3227c
36b5e467e10bed089714b3196334087c8bcc6e5aab9c8206c7899620e9ddd3d6eec2bf263494c8e6f7c99e3c03066d5d3
Decrypted: Hello RSA from Student!
PS C:\Users\hp\Desktop\python workspace>
```

Task 3: Digital Signature:

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

# Generate RSA key pair
key = RSA.generate(2048)
public_key = key.publickey()

# Original message
message = "Hello! This is my digital signature demo."
message_bytes = message.encode()

# Hash the message
hash_value = SHA256.new(message_bytes)

# Sign the hash using private key
signature = pkcs1_15.new(key).sign(hash_value)

print("\n---- Digital Signature ----")
print("Original Message:", message)
print("Signature (hex):", signature.hex())

# Verification – correct message
try:
    pkcs1_15.new(public_key).verify(hash_value, signature)
    print("Signature Verification: SUCCESS")
except:
    print("Signature Verification: FAILED")

# Modify message and test verification again
fake_message = "Hello! This message is changed."
fake_hash = SHA256.new(fake_message.encode())

print("\nTesting After Modifying Message:")
try:
    pkcs1_15.new(public_key).verify(fake_hash, signature)
    print("Verification (modified): SUCCESS – ERROR")
except:
    print("Verification (modified): FAILED (as expected)")
```

Explanation

1. Generate RSA key pair:

The private key is used to create signatures.

The public key is used to verify them.

2. Hash the message using SHA-256:

Signatures do not sign the full message directly.

Instead, they sign a fixed-length hash of the message.

Hashing ensures:

- Fast signing
- Integrity check
- Unique fixed-size output

3. Sign the hash using the private key:

Only the owner of the private key can generate this signature.

The output is a binary signature.

4. Verify using the public key:

Anyone can check the signature with the public key.

If verification succeeds, it proves:

- The message is original
- The message was not modified
- The signer is authentic

5. Modify the message and test:

When even one character is changed, the hash changes completely, so verification **fails**.

This shows that signatures protect message integrity.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  +
---- Digital Signature ----
Original Message: Hello! This is my digital signature demo.
Signature (hex): 55e89a8a4c9af7967e3812b0879fde14da4bd3dc39b73e6d0fb6942ab765b359552a745b9a10d91f8c7b82e3ab9
40fc56beb22be956966545f23152a6ddf6d167faabf6ba0b72c6b8b1f241439b85c0f53d677a9a8157e305edc3323eeeb0cc058376b7
0a4ee08ead995435c7d21298e00baf6d3d86d40c56e442e617c0fe4210bb14112b00fced62cdea848ccf61f88c466db8afba9a60c80f
e46a4f2cc2658ac488f43e853932db628538f46d5c3a3c12bd18d343db241dc59e734eb229b06611dfb5be9c93fe19cc51d819eeac0e
4bb210249ec2dd1e52a956bde2a6beeac1646c171b3dcd9574690512be4de7bc93b6335138a6a4eb4b88c165da900e432
Signature Verification: SUCCESS

Testing After Modifying Message:
Verification (modified): FAILED (as expected)
PS C:\Users\hp\Desktop\python_workspace>
```